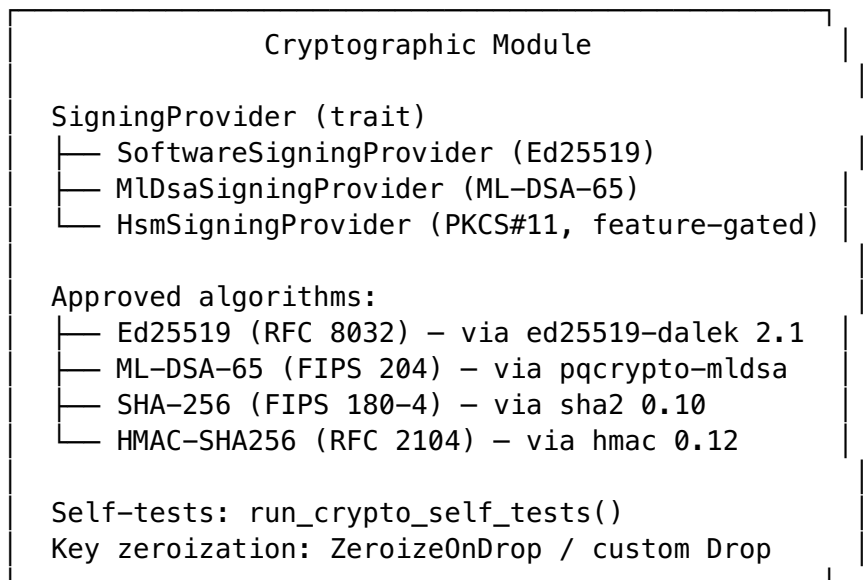


FIPS 140-3 Cryptographic Module Design

This document describes the cryptographic module boundary, approved algorithms, key management, and self-test mechanisms as preparation for FIPS 140-3 validation.

Module boundary

The cryptographic module is contained in `src/identity/signing.rs`. All cryptographic operations (key generation, signing, verification) flow through the `SigningProvider` trait. No other code in the project performs raw cryptographic operations on signing keys.



What is inside the module

Component	File	Purpose
SigningProvider trait	<code>src/identity/signing.rs</code>	Algorithm-agnostic signing interface
SoftwareSigningProvider	<code>src/identity/signing.rs</code>	Ed25519 keypair, sign, verify

Component	File	Purpose
MlDsaSigningProvider	src/identity/signing.rs	ML-DSA-65 keypair, sign, verify
HsmSigningProvider	src/identity/hsm.rs	PKCS#11 delegation (feature-gated)
run_crypto_self_tests()	src/identity/signing.rs	Power-up KAT for all algorithms
SHA-256 hashing	sha2 crate (dependency)	Block hashing, merkle roots
HMAC-SHA256	hmac crate (dependency)	Oracle report authentication

What is outside the module

- TLS (handled by rustls — separate module, not in scope)
- Random number generation (delegated to OS via OsRng / get random)
- Key storage and persistence (RocksDB stores serialized key bytes)
- Application logic (endorsement validation, block creation, API handlers)

Approved algorithms

Algorithm	Standard	Purpose	Implementation
Ed25519	RFC 8032	Digital signatures (classical)	ed25519-dalek 2.1
ML-DSA-65	FIPS 204	Digital signatures (post-quantum, Level 3)	pqcrypto-mldsa 0.1.2 (PQClean reference)
SHA-256	FIPS 180-4	Hashing	sha2 0.10
HMAC-SHA256	RFC 2104 / FIPS 198-1	Message authentication	hmac 0.12

Key management

Key generation

- Ed25519: `SigningKey::generate(&mut OsRng)` — 32 bytes from OS CSPRNG
- ML-DSA-65: `pqcrypto_mldsa::mldsa65::keypair()` — internal CSPRNG from PQClean reference

Key storage

- In-memory during process lifetime

- Serializable via `public_key()` (public) and `from_keys(pk, sk)` (restore from bytes)
- Persistent storage is external to the module (RocksDB or environment)

Key zeroization

Provider	Mechanism
SoftwareSigningProvider	ed25519_dalek::SigningKey implements ZeroizeOnDrop — automatic on drop
MlDsaSigningProvider	Custom Drop replaces secret key with fresh keypair (opaque C struct cannot be directly zeroized)

Key selection

- Runtime selection via `SIGNING_ALGORITHM` environment variable
- One signing key per node lifetime (generated at startup)
- No key rotation during runtime (restart required for new key)

Power-up self-tests (Known Answer Tests)

`run_crypto_self_tests()` executes at node startup before any external data is processed. The node will not start if any test fails.

Test vectors

Algorithm	Test	Verification
Ed25519	Generate key, sign "FIPS-140-3-KAT-Ed25519", verify	Signature verifies correctly
Ed25519	Corrupt one byte of signature, verify	Verification rejected
ML-DSA-65	Generate key, sign "FIPS-140-3-KAT-ML-DSA-65", verify	Signature verifies correctly
ML-DSA-65	Corrupt one byte of signature, verify	Verification rejected
SHA-256	Hash "FIPS-140-3-KAT-SHA256"	Digest matches 11ffe3ed...afb80d0

Failure behavior

If any self-test fails, `main()` panics with:

FATAL: cryptographic self-tests failed – node cannot start

No cryptographic operations are performed before the self-tests complete.

Security levels

Algorithm	NIST Security Level	Equivalent classical security
Ed25519	Not NIST-approved (RFC 8032)	128-bit
ML-DSA-65	Level 3	AES-192 equivalent against quantum
SHA-256	Level 1 (collision), Level 2 (preimage)	128-bit collision resistance

Operational environment

- **OS:** Linux (Docker), macOS (development)
 - **Compiler:** Rust (nightly toolchain)
 - **Random source:** OS-provided CSPRNG (getrandom crate → /dev/urandom on Linux)
 - **No hardware module** unless hsm feature is enabled (PKCS#11)
-

Gap analysis for FIPS 140-3 submission

Requirement	Status	Gap
Cryptographic module boundary	Done	—
Approved algorithms only	Done	Ed25519 is not NIST-approved (would need ML-DSA-65 only mode)
Power-up self-tests (KAT)	Done	Test vectors should be NIST CAVP vectors for formal submission
Key zeroization	Done	ML-DSA-65 uses key replacement (not byte-level zeroing)
Conditional self-tests	Not done	Algorithm-specific tests on first use
Physical security	N/A	Software-only module (Level 1)
	Partial	

Requirement	Status	Gap
Entropy source documentation		Relies on OS CSPRNG; formal entropy assessment needed
Roles and services	Not done	No operator/user role distinction in the module
Formal documentation package	Not done	Security Policy, Finite State Model, design docs required

Estimated readiness: ~60% for FIPS 140-3 Level 1 (software module)